use the `BackgroundService` helper class to create tasks that run on a timer and how to manage your DI lifetimes correctly in a long-running task.

In section 34.2 we'll take the background service concept one step further to create headless worker services using the generic `IHost`. Worker services don't use Razor Pages, API controllers, or minimal API endpoints; instead, they consist only of `IHostedService` services running tasks in the background. You'll also see how to configure and install a worker service app as a Windows Service or as a Linux daemon.

In section 34.3 I introduce the open-source library Quartz.NET, which provides extensive scheduling capabilities for creating background services. You'll learn how to install Quartz.NET in your applications, create complex schedules for your tasks, and add redundancy to your worker services using clustering.

Before we get to more complex scenarios, we'll start by looking at the built-in support for running background tasks in your apps.

34.1 Running background tasks with IHostedService

In most applications, it's common to create tasks that happen in the background rather than in response to a request. This could be a task to process a queue of emails, handling events published to some sort of a message bus or running a batch process to calculate daily profits. By moving this work to a background task, your user interface can stay responsive. Instead of trying to send an email immediately, for example, you could add the request to a queue and return a response to the user immediately. The background task can consume that queue in the background at its leisure.